

SATURDAY, SEPTEMBER 26 · AFTERNOON

10 Retrieval augmented generation · 11 Open source models

Kerry Back

This afternoon

Retrieval augmented generation

Answering from your own documents, with a citation you can open.

Open source models

What you can download and run yourself, and how far behind it is.

10 · RETRIEVAL AUGMENTED GENERATION

Answering from your own documents, with a citation you can open.

The other eighty per cent

YESTERDAY

Rows, columns, joins. Five spreadsheets and a total you could reconcile.

EVERYTHING ELSE A FIRM HOLDS

Contracts, mail, board papers, policies, reports, decks, notes.

65% of data leaders call their structured data ready for AI. 39% say the same of the unstructured kind, which is most of the volume.

Why not just paste it in

It was not trained on yours

Your contracts were not on the public web. Nothing in the weights knows them.

The window is finite

A million tokens is large and your document store is larger.

You pay for every turn

Input is billed each turn. Pasting the corpus into every question bills the corpus every question.

Four steps

- 1 Chunk. Split the documents into passages of a few hundred words.
- 2 Embed. Turn each passage into a vector, the same way tokens became vectors on Thursday.
- 3 Retrieve. Embed the question and find the handful of passages closest to it.
- 4 Paste. Put those passages in the prompt, with the question.

The model never reads your corpus. It reads an excerpt that a similarity search chose.

A passage becomes 384 numbers

An embedding model reads a passage and returns a fixed-length list of numbers — 384, or 768, or 1,536, depending on the model.

Passages about the same thing land near each other, whatever words they used. “Revenue fell in the third quarter” sits close to “Q3 sales were down”, and a keyword search for “revenue” finds only the first.

This is why retrieval works on a question phrased in the reader’s words rather than the author’s.

Chunking is the design decision

CHOICE	WHAT IT COSTS
Chunks too small	the passage arrives without the sentence that made it mean something
Chunks too large	the relevant line is buried in a page of irrelevance you paid for
Split mid-table	numbers arrive with no column headings
No overlap	an answer spanning a boundary is never retrieved whole

Split on the document's own structure — sections, clauses, slides — before falling back on a word count.

Two ways to have one

A HOSTED TOOL

Upload the folder, ask questions.
Somebody else chose the chunking, the embedder and the number of passages retrieved.

A PIPELINE YOU OWN

Your chunking rules, your embedder, your store, and the retrieved text visible before it reaches the model.

Start hosted. Build your own when the answers are wrong and you need to see which passages came back.

Build one over your own documents

EXERCISE 16 · 60 MINUTES

- 1 Put twenty or more of your own documents in a folder — reports, policies, decks, anything real but not confidential.
- 2 Have them chunked and embedded into a local store, and have the chunking rule stated in writing.
- 3 Add a search tool to an agent, so it retrieves before it answers.
- 4 Ask five questions you already know the answers to.
- 5 Make it print the passages it retrieved, not just the answer.

Step 5 is the whole exercise. An answer you cannot trace is the thing RAG was supposed to fix.

Make it cite, then check the citation

A citation is only worth having if somebody opens it.

- 1 Ask a question whose answer is in exactly one document.
- 2 Ask one whose answer is spread over three.
- 3 Ask one whose answer is in none of them, and see whether it says so.

The third is the one that fails. A retriever always returns its closest matches, whether or not any of them are relevant.

Permissions do not survive chunking

THE DOCUMENT

Sat in a folder with an access list.
Three people could open it.

THE CHUNK IN THE STORE

Is a row in a database with a vector
next to it. Usually carries no access list
at all.

Index everything and the retriever will hand a new analyst a paragraph from a file they could never have opened.

Retrieved text is untrusted text

A poisoned document in the index is a prompt injection with a delivery mechanism.

Anything that reaches the model reaches it as text, and text from a retrieved passage arrives in the same stream as your question. Whoever can put a document into the index can put instructions in front of the model.

Which is the Sunday morning question again: who can write to the corpus, and what can the agent do once it has read one.

When it is the wrong tool

Few enough to paste

Twenty pages fit in the window. Retrieval adds a failure mode and buys nothing.

The answer is a number

Totals, counts and rankings come from a query against the data, not a similarity search over prose.

The corpus changes hourly

Every change means re-embedding. A live query beats a stale index.

Break your own

EXERCISE 17 · 40 MINUTES

- 1 Add a document to your index that contradicts another one. Ask the question both answer.
- 2 Add one containing an instruction addressed to the agent. Ask an unrelated question.
- 3 Ask something answered only by a table you chunked in the middle.
- 4 Write down which of the three your setup handled, and what you would change.

- What you indexed, and how you chunked it
- One question it answered well, with the passage it used
- The one that broke it

Screen shares

11 · OPEN SOURCE MODELS

What GPQA Diamond measures

Four-way multiple-choice questions in biology, physics and chemistry, written by people who hold or are working toward a PhD in the field they wrote for.

25%

GUESSING

34%

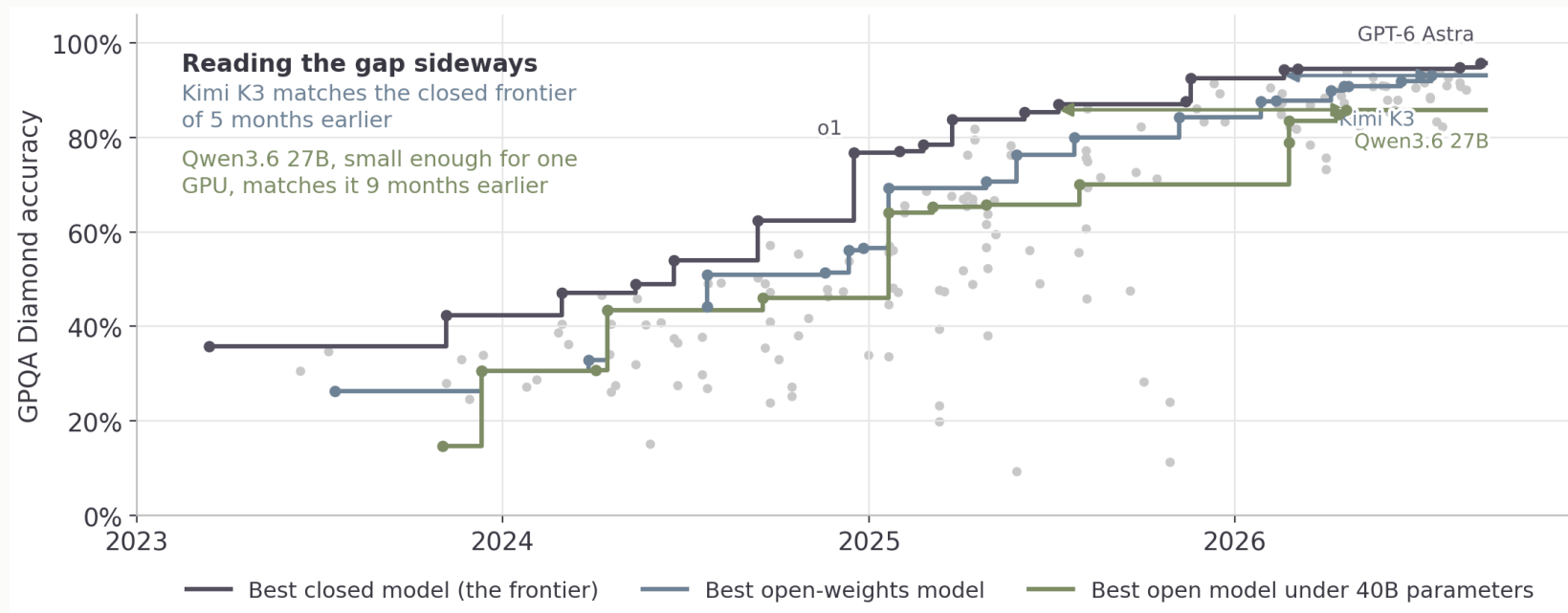
SKILLED NON-EXPERTS, 30+
MINUTES OF OPEN WEB SEARCH

74%

PHDS ANSWERING INSIDE THEIR
OWN FIELD

Diamond is the hardest 198 of the 448 questions: both expert annotators answered correctly and most non-experts did not. Rein et al., 2023.

How far behind is it?



Each line is that tier's best score so far; gray dots are every other model. Source: Epoch AI, 8 Sep 2026.

What Terminal-Bench measures

An agent is given a container, a plain-English objective, and a terminal. It has to actually do the job: compile the code, train the model, configure the server, find the bug.

7

TASK DOMAINS: SOFTWARE, ML, SCIENCE, OPERATIONS, SECURITY, HARDWARE, MEDIA

4

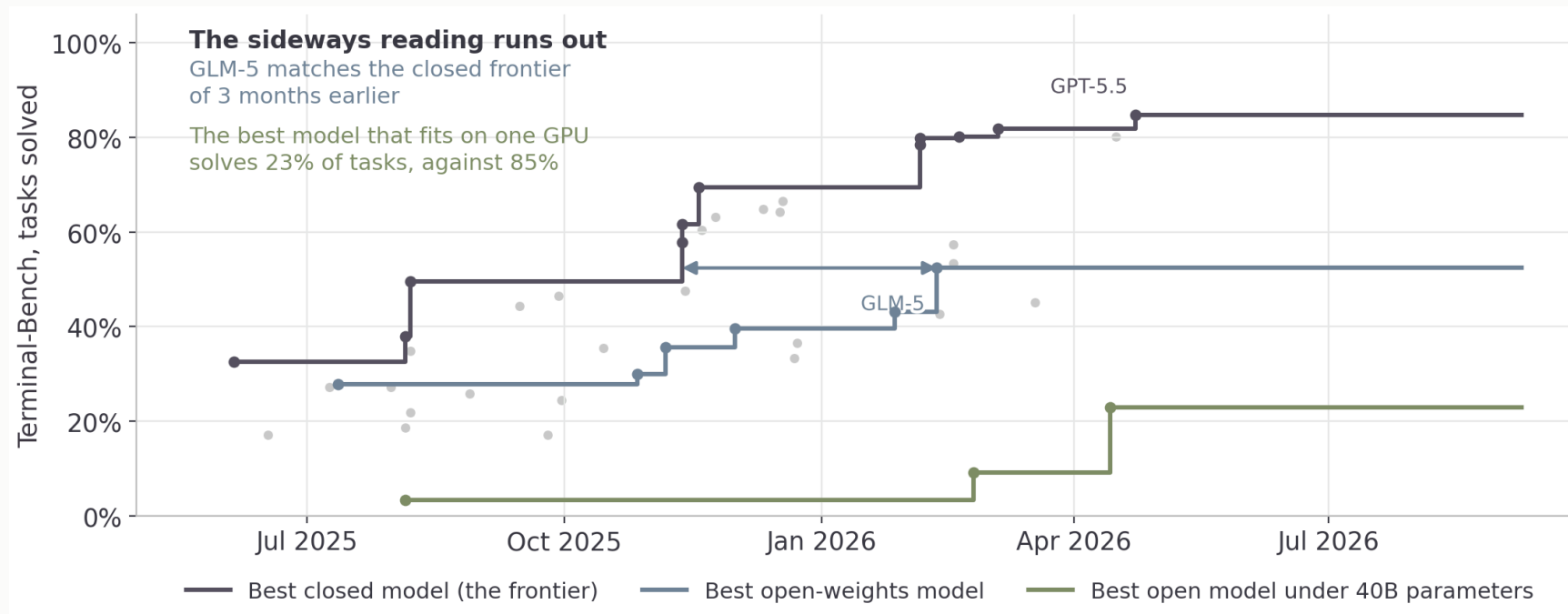
PARTS TO A TASK: INSTRUCTION, CONTAINER, TEST SUITE, REFERENCE SOLUTION

0 or 1

PER TASK — THE TESTS PASS OR THEY DO NOT

Built by the Laude Institute with Stanford researchers. No human baseline is published. The version charted here dates from mid-2025; a harder v4.0 has since cut the frontier to about 60%.

The same three tiers, on agentic work



Epoch's mirror of Terminal-Bench; its closed-model coverage runs to April 2026.

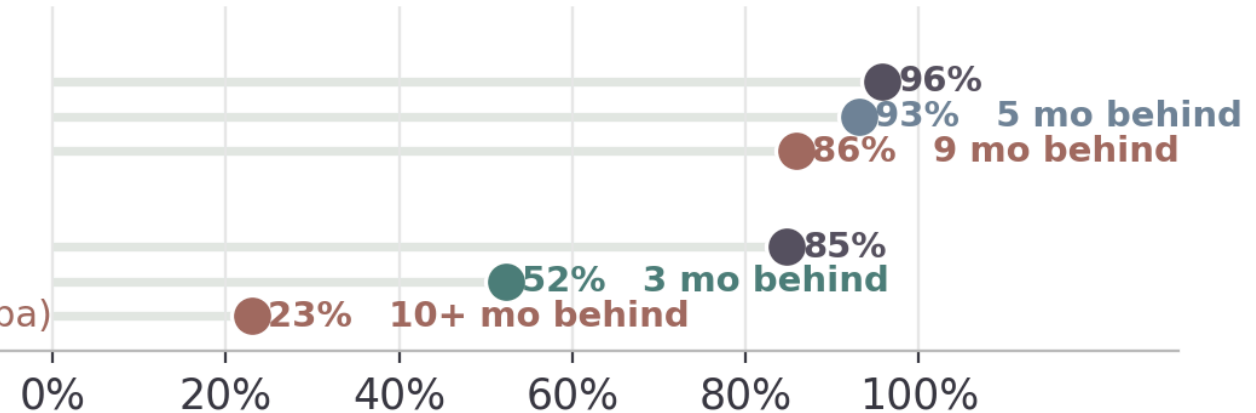
Where each tier stands today

GPQA Diamond

closed GPT-6 Astra (OpenAI)
open weights Kimi K3 (Moonshot)
open, under 40B Qwen3.6 27B (Alibaba)

Terminal-Bench

closed GPT-5.5 (OpenAI)
open weights GLM-5 (Z.ai)
open, under 40B Qwen 3.6 35B-A3B (Alibaba)



Best score in each tier, September 2026. "Behind" is how long ago the closed frontier was at that score. Under 40B parameters is what a 32 GB consumer GPU holds at 4-bit; a 128 GB Mac holds several times more.

Two numbers, two different answers

THE MONTHS

Three to five months, on both benchmarks. A model small enough for one consumer GPU is nine to ten months back, on both.

THE SCORES

On graduate science questions the open gap is 3 points. On agentic terminal work it is 33, and the small model solves 23% against the frontier's 85%.

GPQA Diamond is near its ceiling, which compresses the gap: five months of lag is worth three points because there is nowhere left to go. Terminal-Bench is not near any ceiling, and there the same few months is most of the task.

Answer from knowledge

A 27B open model scores 86% on GPQA Diamond against the frontier's 96%, on hardware you own.

Reason over your documents

A large open model is a few months behind and self-hostable. Test it on your own task before taking the ranking.

Run a multi-step agent

On Terminal-Bench the best open model solves 52% and the best small one 23%, against the frontier's 85%.

What that means for a choice

Not every job needs the largest model

	WHERE IT RUNS	WHAT IT SUITS
A frontier model	the provider's API	judgment, long context, code
A mid-size model	the same API, less per token	classification, extraction, routing
A small open model	your own hardware	high volume, or data that cannot leave
A fine-tuned small model	your own hardware	one narrow task, done constantly

On a single question the price difference is negligible. On a million classifications it is the budget.

Distillation, in one paragraph

Run the frontier model on a few thousand examples of your task. Keep the inputs and its outputs. Train a small model on those pairs.

The small model does not become generally capable. It becomes capable at the one task you sampled, at a fraction of the cost per call, on hardware you control.

Everything so far has run inside a tool somebody else built. Tomorrow you go under it, and then ask what it would take to let one loose on real data.

Tomorrow